

TEMA 096. PROCESOS DE PRUEBAS Y GARANTÍA DE CALIDAD EN EL DESARROLLO DE SOFTWARE. PLANIFICACIÓN, ESTRATEGIA DE PRUEBAS Y ESTÁNDARES. NIVELES, TÉCNICAS Y HERRAMIENTAS DE PRUEBAS DE SOFTWARE. CRITERIOS DE ACEPTACIÓN DE SOFTWARE.

Actualizado a 23/01/2022

1 INTRODUCCIÓN

Para la elaboración de esta documentación se ha tomado como referencia principal los estudios y glosarios de International Software Testing Qualifications Board, en adelante ISTQB: <https://www.istqb.org/> Comité internacional que suministra planes de estudios y glosarios para distintos niveles sobre los que se definen las guías para la acreditación y evaluación de los profesionales del testing. Además, de la norma ISO/IEC 25000 SQuaRE y técnicas y prácticas de Métrica³. Cada título indica el origen de la información.

El proceso de pruebas aplicado en el desarrollo, mantenimiento y operación del software reduce riesgos, contribuye a la calidad del sistema y ayuda a verificar requisitos contractuales, legales y estándares específicos del sector. Las pruebas aportan fiabilidad y calidad software.

Las actividades de pruebas se dan antes y después de la ejecución de la prueba:

- Planificar y controlar.
- Seleccionar las condiciones de las pruebas.
- Diseñar y ejecutar casos de prueba.
- Comprobar resultados.
- Evaluar criterios de salida.
- Elaborar informes sobre el proceso de pruebas y el sistema probado.
- Finalizar o completar actividades de cierre tras finalizar una fase de prueba.
- Revisión de documentos (incluye código fuente).
- Relación de análisis estáticos.

Existen distintos niveles, y tipos de pruebas, dinámicas y estáticas, generando información para mejorar tanto el sistema probado como el desarrollo y los procesos de prueba. Para llevar a cabo estas pruebas existen diferentes herramientas.

El coste de eliminar defectos se incrementa con el tiempo durante el cual el defecto permanece en el sistema. La identificación de errores en etapas tempranas permite la corrección de los mismos a costos reducidos.

Cuanto más temprana es la detección de un defecto, menos costosa es su corrección

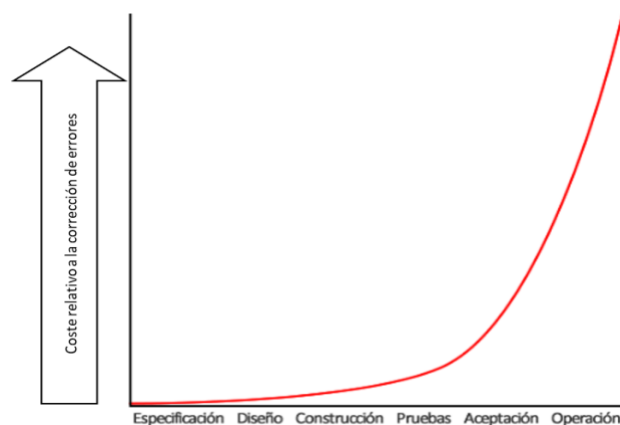


Imagen 1 Gráfica Coste / Momento de detección del error

Conceptos Bug, defecto, error, fallo (ISTQB): Una persona puede cometer un **error**(equivocación) que puede producir un **defecto (falta, bug)** en el código de programa o en un documento. Si se ejecuta un defecto en el código, el sistema puede no hacer lo que se espera o hacer algo que no debería, lo que provocaría un **fallo**.

Error humano → Defecto (Bug) → Fallo
Error agentes externos → Fallo

1.1 PRINCIPIOS DE PRUEBAS (ISTQB)

1.1.1 PRINCIPIO 1 – LAS PRUEBAS DEMUESTRAN LA PRESENCIA DE DEFECTOS

Las pruebas pueden demostrar que hay defectos, pero no pueden probar que no los hay. Las pruebas reducen la probabilidad de que haya defectos ocultos en el software, pero, aunque no se detecte ningún defecto, no constituyen una evidencia de corrección.

1.1.2 PRINCIPIO 2 – LAS PRUEBAS EXHAUSTIVAS NO EXISTEN

Probar todo (todas las combinaciones de entradas y precondiciones) es imposible, salvo en casos triviales. En lugar de pretender hacer pruebas exhaustivas, se deben realizar análisis de riesgos y prioridades para centralizar los esfuerzos de las pruebas.

1.1.3 PRINCIPIO 3 – PRUEBAS TEMPRANAS

Para identificar los defectos en una etapa temprana, las actividades de pruebas se iniciarán lo antes posible en el ciclo de vida del software o del desarrollo del sistema, debiendo centrarse en objetivos definidos.

1.1.4 PRINCIPIO 4 – AGRUPACIÓN DE DEFECTOS

Las pruebas deben concentrarse de manera proporcional en la densidad esperada, y más tarde observada, de los defectos de los módulos. Normalmente la mayor parte de los defectos detectados durante las pruebas previas al lanzamiento y la mayoría de los fallos operativos se concentran en un número reducido de módulos.

1.1.5 PRINCIPIO 5 – PARADOJA DEL PESTICIDA

Si repetimos las mismas pruebas una y otra vez, eventualmente la misma serie de casos de prueba dejará de encontrar defectos nuevos. Para superar esta “paradoja del pesticida”, los casos de prueba deben revisarse periódicamente y deben escribirse pruebas nuevas y diferentes para ejercitar distintas partes del software o del sistema con vistas a poder detectar más defectos.

1.1.6 PRINCIPIO 6 – LAS PRUEBAS DEPENDEN DEL CONTEXTO

Las pruebas se llevan a cabo de manera diferente según el contexto. Así, por ejemplo, la forma de probar un software de seguridad crítico variará de la de un sitio de comercio electrónico.

1.1.7 PRINCIPIO 7 – FALACIA DE AUSENCIA DE ERRORES

La detección y corrección de defectos no servirá de nada si el sistema construido no es usable y no cumple las expectativas y necesidades de los usuarios.

2 ASEGURAMIENTO DE LA CALIDAD Y PROCESOS DE PRUEBAS (ISTQB)

Aseguramiento de la calidad del software (SQA) es un proceso que persigue que todos los procesos, métodos, actividades y elementos de trabajo de ingeniería de software sean observados y cumplan con los estándares definidos.

El proceso de pruebas software es una actividad dentro del proceso de Aseguramiento de la calidad del software (SQA).

La parte más visible del proceso de pruebas es la ejecución de pruebas. No obstante, en el proceso de pruebas son necesarias, al igual que en el proceso de desarrollo, tareas de planificación, análisis, diseño y evaluación y generación de informes. Los planes de pruebas también deben indicar el tiempo necesario para planificar las pruebas, diseñar los casos de prueba, preparar la ejecución y evaluar los resultados.

El proceso básico de pruebas consta de las siguientes actividades principales:

- Planificación y control de las pruebas
- Análisis y diseño de pruebas
- Implementación y ejecución de pruebas
- Evaluación de los criterios de salida y elaboración de informes
- Actividades de cierre de pruebas
- Control de pruebas (en todas las actividades)

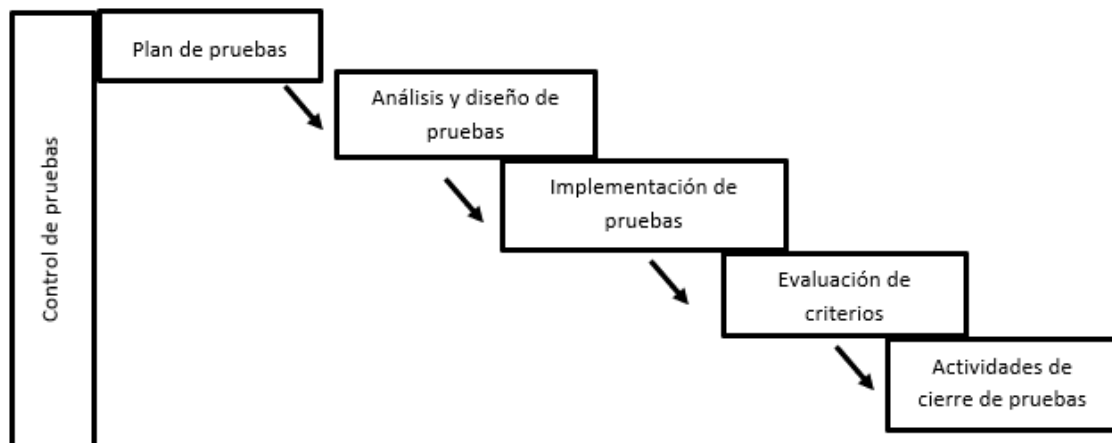


Imagen 2 Proceso de Pruebas (ISTQB)

Las actividades del proceso pueden solaparse o realizarse a la vez. Es necesario adaptar estas actividades al contexto del sistema y del proyecto.

3 PLANIFICACIÓN (ISTQB)

El proceso de pruebas no es un proceso aislado, las actividades de pruebas están asociadas a actividades de desarrollo de software. Distintos modelos de ciclo de vida de desarrollo requieren distintos enfoques de las tareas de validación y verificación del aseguramiento de la calidad.

Conceptos: Verificación y Validación

- **Verificación** Comprobación de la conformidad con los requisitos establecidos (*ISO 9000*). Conjunto de actividades que aseguran que el software implementa correctamente una función específica. (*Pressman*). Da respuesta a la pregunta ¿estamos construyendo el producto correctamente? (*Boehm*).
Busca comprobar que el sistema cumple con los requerimientos especificados (funcionales y no funcionales) ¿El software está de acuerdo con su especificación?
- **Validación** Comprobación de la idoneidad para el uso esperado (*ISO 9000*). conjunto diferente de tareas que aseguran que el software que se construye sigue los requerimientos del cliente. (*Pressman*) Da respuesta a la pregunta ¿estamos construyendo un producto correcto? (*Boehm*).
Busca comprobar que el software hace lo que el usuario espera. ¿El software cumple las expectativas del cliente?

Modelo en V

Es el modelo de desarrollo software más utilizado. Desarrollo y pruebas tienen dos ramas iguales. Cada nivel de desarrollo tiene su correspondiente nivel de pruebas. Las pruebas (rama derecha) se diseñan en paralelo al desarrollo software (rama izquierda). Las actividades del proceso de pruebas tienen lugar a lo largo de todo el ciclo de vida software. Cada nivel de desarrollo se verifica respecto de los contenidos del nivel que le precede.

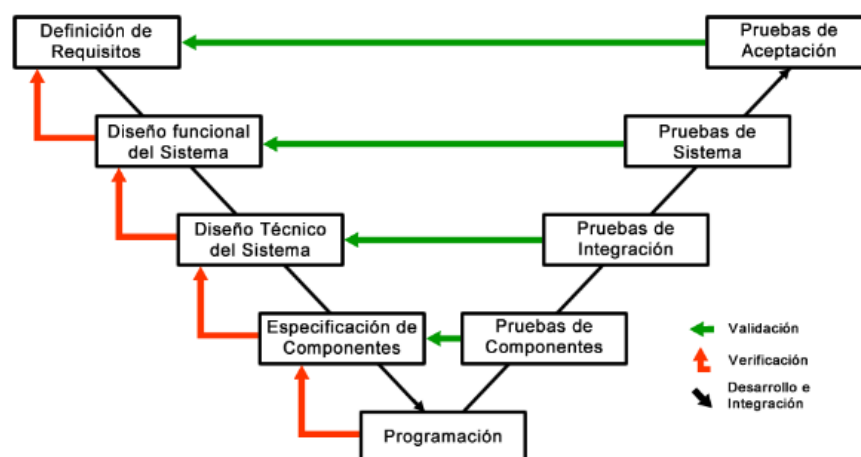


Imagen 3 Modelo en V

Modelo en W

El modelo W puede ser visto como una extensión del modelo en V, con la diferencia de que, en los estados del modelo en W, ciertas actividades de control de calidad se realizarán en paralelo con el proceso de desarrollo.

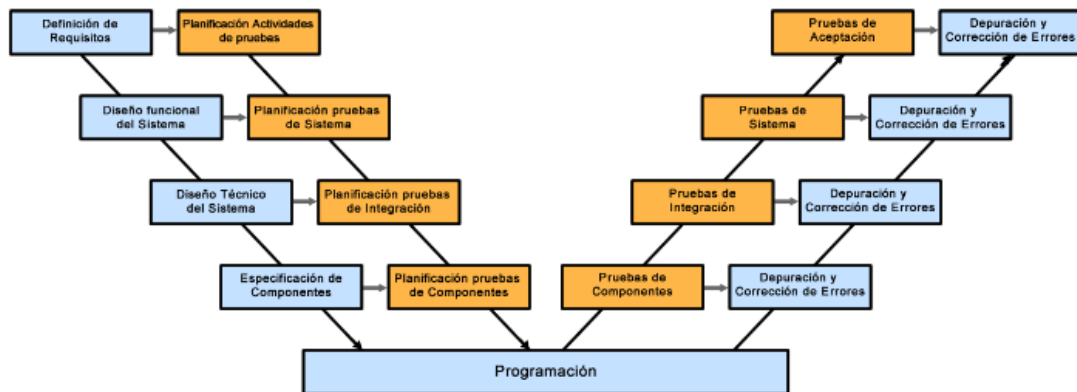


Imagen 4 Modelo en W

En los ciclos de vida Iterativo- incremental, Prototipo, RAD, RUP o Agile, la planificación de las tareas de verificación y validación deberá adaptarse ya que el producto es diseñado y construido mediante muchas entregas e iteraciones.

La planificación de pruebas comienza al inicio de un proyecto de desarrollo y se ajusta a lo largo del ciclo de vida del proyecto. La planificación de pruebas también cubre la creación y actualización del plan de pruebas. Las siguientes actividades forman parte de la planificación de pruebas:

- Estrategia de pruebas (test strategy).
- Planificación de recursos (resource planning).
- Prioridad de las pruebas (priority of tests).
- Soporte de herramientas (tool support).

4 ESTRATEGIA

Existen distintas prácticas de aplicación del proceso de pruebas software que afectan directamente al proceso de desarrollo.

4.1 TDD (TEST DRIVEN DEVELOPMENT)

TDD o desarrollo guiado por pruebas de software, o Test-driven development (TDD) es una práctica de ingeniería de software que involucra otras dos prácticas:

- Escribir las pruebas primero (Test First Development) basándose en los requisitos y se verifica que las pruebas fallen.
- Escribir el código fuente que pase la prueba satisfactoriamente.
- Refactorización del código escrito (Refactoring).

Para escribir las pruebas generalmente se utilizan las pruebas unitarias. En primer lugar, se escribe una prueba y se verifica que las pruebas fallan. A continuación, se implementa el código que hace que la prueba pase satisfactoriamente y seguidamente se refactoriza el código escrito.

El propósito del desarrollo guiado por pruebas es lograr un código limpio que funcione. La idea es que los requisitos sean traducidos a pruebas, de este modo, cuando las pruebas pasen se garantizará que el software cumple con los requisitos que se han establecido.

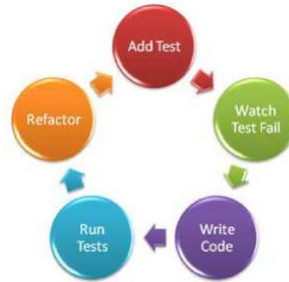


Imagen 5 Ciclo TDD

4.1.1 VENTAJAS DE TDD

- **Más calidad y reducción de tiempos a medio y largo plazo:** a pesar de los elevados requisitos iniciales de aplicar esta metodología, el desarrollo guiado por pruebas (TDD) puede proporcionar un gran valor añadido en la creación de software, produciendo aplicaciones de más calidad y en menos tiempo.
- **Guía el desarrollo:** ofrece más que una simple validación del cumplimiento de los requisitos, una ayuda a guiar el diseño de un programa.
- **Análisis de requisitos:** al hacer primero las pruebas se realiza un ejercicio previo de análisis en profundidad de los requisitos y de los diversos escenarios.
- **Código creado mínimo necesario:** el hecho que sólo se implemente el código necesario para resolver un caso de prueba, hace que el código creado sea el mínimo necesario, reduciendo redundancia y bloques de código de “por si acaso”.

4.2 BDD (BEHAVIOR DRIVEN DEVELOPMENT)

BDD (Behavior Driven Development), es una estrategia de desarrollo dirigido por comportamiento, que ha evolucionado desde TDD (Test Driven Development), aunque no se trata de una técnica de pruebas como tal.

A diferencia de TDD, BDD se define en un idioma común entre todos los stakeholders, lo que mejora la comunicación entre equipos tecnológicos y no técnicos. Tanto en TDD como en BDD, las pruebas se deben definir antes del desarrollo, aunque en BDD, las pruebas se centran en el usuario y el comportamiento del sistema, a diferencia del TDD que se centra en funcionalidades.

4.2.1 CARACTERÍSTICAS BDD

La principal diferencia es que todas las definiciones BDD se escriben en un idioma común. El principal objetivo es que el equipo describa los detalles de cómo se debe comportar la aplicación a desarrollar, y de esta forma será comprensible por todos.

A diferencia de BDD, TDD funciona apropiadamente siempre que las habilidades técnicas de la dirección sean lo suficientemente sólidas, y no siempre es así.

En estas circunstancias, la BDD tiene la ventaja de que las pruebas se pueden escribir en un lenguaje común utilizado por todas las partes interesadas.

Cada requisito debe convertirse en historias de usuario, definiendo ejemplos concretos. Cada ejemplo debe ser un escenario de un usuario en el sistema.

Requiere utilizar la fórmula 'Given-When-Then' u otras como las historias de usuario 'Role-Feature-Reason'.

'Given-When-Then'

- Given 'dado': Se especifica el escenario, las precondiciones.
- When 'cuando': Las condiciones de las acciones que se van a ejecutar.
- Then 'entonces': El resultado esperado, las validaciones a realizar.

'Role-Feature-Reason'

- As a '**Como**': Se especifica el tipo de usuario.
- I want '**deseo**': Las necesidades que tiene.
- So that '**para que**': Las características para cumplir el objetivo.

4.2.2 HERRAMIENTAS DE DEFINICIÓN BDD

La herramienta más destacada, basado en el patrón 'Given-When-Then' es Cucumber, un framework de test con soporte para BDD. En Cucumber, las especificaciones de BDD están escritas en lenguaje Gherkin, basado en 'Given-When-Then'.

Hay muchas otras herramientas, tantas como lenguajes de programación:

- Java: JBehave, JDave, Instinct, beanSpec
- C: CSpec
- C#: NSpec, NBehave
- .NET: NSpec, NBehave, SpecFlow
- PHP: PHPSpec, Behat
- Ruby: RSpec, Cucumber
- JavaScript: JSSpec, jspec
- Python: Freshen

4.2.3 VENTAJAS DE BDD (BEHAVIOR DRIVEN DEVELOPMENT)

- Ya no estás definiendo 'pruebas', sino que está definiendo 'comportamientos'.
- Mejora la comunicación entre desarrolladores, testers, usuarios y la dirección.
- Debido a que BDD se especifica utilizando un lenguaje simplificado y común, la curva de aprendizaje es mucho más corta que TDD.
- Como su naturaleza no es técnica, puede llegar a un público más amplio.
- El enfoque de definición ayuda a una aceptación común de las funcionalidades previamente al desarrollo.
- Esta estrategia encaja bien en las metodologías ágiles, ya que en ellas se especifican los requisitos como historias de usuario y de aceptación.

4.3 AUTOMATIZACIÓN DE PRUEBAS Y TESTING CONTINUO

Realizar pruebas continuas (Continuous Testing) es tanto una práctica como una filosofía dentro de los equipos de desarrollo de software. Los desarrolladores y los especialistas en garantizar la calidad de las aplicaciones inician la práctica del testing continuo cuando mediante un Pipeline CI/CD (Continuous Integration / Continuous Deployment), activan una lista de pruebas automatizadas que se ejecutan con cada construcción y entrega.

La organización debe establecer una estrategia de testing que defina un enfoque óptimo, una estrategia de implementación de pruebas y funciones de apoyo continuas. Se deben identificar las técnicas de análisis estático a utilizar, a través de analizadores de código como SonarQube o compiladores y técnicas de análisis dinámico a aplicar en la elaboración de pruebas unitarias (JUnit), de integración (SOAPUI, POSTMAN), de sistema (JMETER), de aceptación (Cucumber) o de regresión (Selenium, Cypress).

4.3.1 AUTOMATIZACIÓN DE PRUEBAS

Podemos definir la automatización de pruebas como la automatización de la garantía de calidad para un software, desarrollo o servicio.

La Garantía de calidad (también denominada en inglés como Quality Assurance (QA)) es la encargada de gestionar la calidad del proyecto en todas sus fases. A diferencia con el tester, el QA se encuentra presente en todas las partes del proyecto, es decir, se encuentra en comunicación tanto con desarrolladores como con la dirección o la gestión del producto. El testing puede englobarse dentro del proceso de QA, como una parte de él.

A través del proceso de QA automatizado, el profesional encargado garantiza un trabajo ágil y rápido, con la mayor calidad en los procesos, equipos y en el resultado final, gracias a que automatizando las comprobaciones se optimizan las pruebas, rendimientos y se hacen mejoras continuas.

Ejemplos de herramientas de automatización son QMetry, Worksoft, Lambdatest, Ranorex, TestComplete, Qualibrate, etc.

4.3.2 TESTING CONTINUO

Testing continuo es una forma óptima de implementar una estrategia de pruebas específicas, ya que proporciona a los desarrolladores una retroalimentación antes de que el código llegue a un entorno de entrega.

Las pruebas continuas tienen que ser automatizadas primero, integradas en el pipeline de CI/CD y configuradas con alertas para que las herramientas notifiquen los problemas detectados. Continuous Testing incluye, además, pruebas para automáticas que permiten detectar problemas en producción.

Las herramientas de pruebas actuales se integran con herramientas CI /CD como Jenkins, CircleCI, Bamboo. En función del nivel de prueba a automatizar existen diferentes herramientas como SmartBear, BlazeMeter, Tricentis qTest, BrowserStack, SauceLabs, Postman, SOAPUI, Selenium, Cypress, JUnit o analizadores de código como SonarQube.

4.3.3 VENTAJAS

- El tiempo de ejecución de pruebas se reduce de forma significativa, aumentando la **velocidad y eficiencia** en el uso de recursos.
- Al poder abarcar más caminos de manera automatizada, cubre una amplia variedad del código generado, mejorando así el resultado final del software.
- Son **reutilizables** en procesos de Integración Continua.
- Es capaz de manejar grandes volúmenes de datos, lo que permite probar diferentes escenarios, siendo así **escalable**.
- Evita el error humano en pruebas repetitivas, apostando por la **consistencia y objetividad**.

5 ESTÁNDARES

Existen multitud de estándares relacionados con el proceso de pruebas y la garantía de calidad en el desarrollo de software. Se muestra una evolución de los más relevantes:

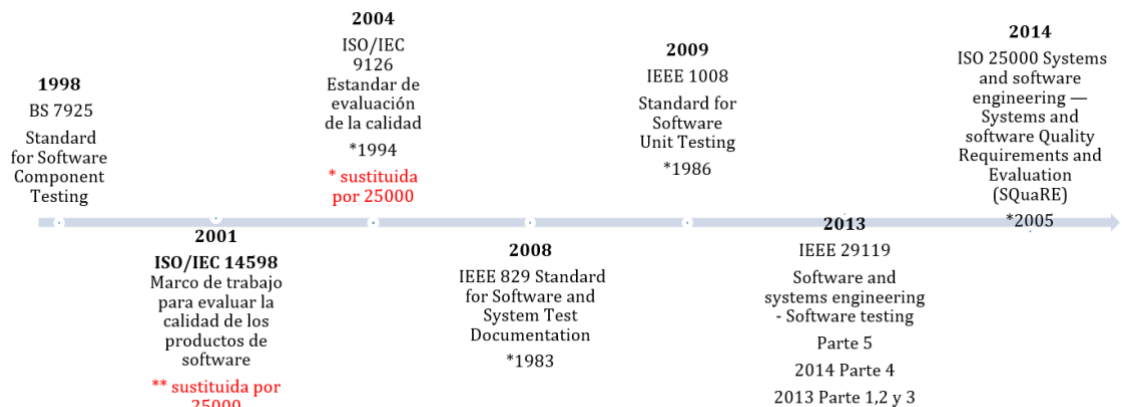


Imagen 6 Evolución Estándares de Pruebas Software y Calidad Software

Hay que destacar el conjunto de normas SQuaRE, System and Software Quality Requirements and Evaluation, ISO/IEC 25000, publicado en 2005 y actualizado en 2014, que sustituye al estándar internacional de evaluación de la calidad ISO 9126, publicado en 1994 y actualizado en 2004.

Además, existen multitud de marcos de referencia o buenas prácticas que guardan relación con los procesos de pruebas y garantía de calidad en el desarrollo de software, como:

- Moprosoft: modelo de Referencia de Procesos conformado por un conjunto de buenas prácticas y procesos de gestión e ingeniería de software.
- CMMI, Capability Maturity Model Integration, conjunto de buenas prácticas organizadas por capacidades críticas de negocio con el objetivo de mejorar su rendimiento.
- SWEBOK, Software Engineering Body of Knowledge, es un documento creado por la Software Engineering Coordinating Committee, promovido por el IEEE.
- ITIL, Information Technology Infrastructure Library, biblioteca de infraestructura y tecnologías de la información, es un conjunto de conceptos y mejores prácticas referentes a la gestión de servicios TI.
- ISTQB, International Software Testing Qualifications Board, comité Internacional de Certificaciones de Pruebas de Software.

- Métrica 3, Metodología de Planificación, Desarrollo y Mantenimiento de Sistemas de Información creada por el Ministerio de Administraciones Públicas, que contempla prácticas de pruebas y una interfaz para el aseguramiento de la calidad.

5.1 ISO/IEC /IEEE 29119 SOFTWARE TESTING STANDARD

Su objetivo es cubrir todo el ciclo de vida de las pruebas de sistemas software incluyendo los aspectos relativos a la organización, gestión, diseño y ejecución de las pruebas, para remplazar varios estándares IEEE y BSI sobre pruebas de software. Su estructura consta de cinco partes, un estándar relativo a revisiones y un modelo de evaluación basado en los procesos de la parte 2:

- **Conceptos y definiciones:** ISO/IEC/IEEE 29119-1:2013 Software and systems engineering - Software testing - Part 1: Concepts and definitions.
- **Procesos de prueba** ISO/IEC/IEEE 29119-2:2013 Software and systems engineering - Software testing - Part 2: Test processes.
- **Documentación de prueba** ISO/IEC/IEEE 29119-3:2013 Software and systems engineering - Software testing - Part 3: Test documentation
- **Técnicas de prueba** ISO/IEC/IEEE 29119-4:2015 Software and systems engineering - Software testing - Part 4: Test techniques.
- **Pruebas conducidas por palabras clave** ISO/IEC/IEEE 29119-5:2016 Software and systems engineering - Software testing - Part 5: Keyword-Driven Testing.
- **Revisiones** ISO/IEC 20246:2017 Software and Systems Engineering - Work Product Reviews.
- **Modelo de evaluación en los procesos de prueba (parte 2)** ISO/IEC 33063:2015 Information technology - Process assessment - Process assessment model for software testing.

5.2 ISO 25000:2014

El conjunto de normas ISO 25000:2014 conocidas como SQuaRE (System and Software Quality Requirements and Evaluation) (Evaluación de Calidad de Productos Software), tiene como objetivo general organizar, enriquecer y unificar las series que cubren dos procesos principales:

- Especificación de requisitos de calidad del software.
- Evaluación de la calidad del software, soportada por el proceso de medición de calidad del software.

El conjunto de normas ISO 25000:2014 provienen de la **evolución de las normas ISO/IEC 9126**, estándar de evaluación de la calidad de un software, e **ISO/IEC 14598**, marco de trabajo para evaluar la calidad del software.

Se divide en 6 secciones: ISO/IEC 2500n, ISO/IEC 2501n, ISO/IEC 2502n, ISO/IEC 25030, ISO/IEC 25040, e ISO/IEC 25050-25099.

El contenido de la ISO/IEC 25000 está compuesto por: definiciones de conceptos, modelos o patrones de referencia, guía completa de cada división, y listado de los estándares internacionales.

El **modelo de calidad** representa el sistema para la evaluación de la calidad del producto y determina las **características de calidad** que se van a tener en cuenta a la hora de evaluar las propiedades de un producto software determinado.

La calidad del producto software se puede interpretar como el grado en que dicho producto satisface los requisitos de sus usuarios aportando de esta manera un valor. Son precisamente estos requisitos (funcionalidad, rendimiento, seguridad, mantenibilidad, etc.) los que se encuentran representados en el modelo de calidad, el cual categoriza la calidad del producto en características y subcaracterísticas.

El modelo de calidad del producto definido por la ISO/IEC 25010 se encuentra compuesto por las ocho características de calidad que se muestran en la siguiente figura:



Imagen 7 Características del producto Software - ISO/IEC 25010 Fuente: iso25000.com

5.2.1 ADECUACIÓN FUNCIONAL

Representa la capacidad del producto software para proporcionar funciones que satisfacen las necesidades declaradas e implícitas, cuando el producto se usa en las condiciones especificadas. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Completitud funcional.** Grado en el cual el conjunto de funcionalidades cubre todas las tareas y los objetivos del usuario especificados.
- **Corrección funcional.** Capacidad del producto o sistema para proveer resultados correctos con el nivel de precisión requerido.
- **Pertinencia funcional.** Capacidad del producto software para proporcionar un conjunto apropiado de funciones para tareas y objetivos de usuario especificados.

5.2.2 EFICIENCIA DE DESEMPEÑO

Esta característica representa el desempeño relativo a la cantidad de recursos utilizados bajo determinadas condiciones. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Comportamiento temporal.** Los tiempos de respuesta y procesamiento y las ratios de *throughput* de un sistema cuando lleva a cabo sus funciones bajo condiciones determinadas en relación con un banco de pruebas (*benchmark*) establecido.
- **Utilización de recursos.** Las cantidades y tipos de recursos utilizados cuando el software lleva a cabo su función bajo condiciones determinadas.
- **Capacidad.** Grado en que los límites máximos de un parámetro de un producto o sistema software cumplen con los requisitos.

5.2.3 COMPATIBILIDAD

Capacidad de dos o más sistemas o componentes para intercambiar información y/o llevar a cabo sus funciones requeridas cuando comparten el mismo entorno hardware o software. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Coexistencia.** Capacidad del producto para coexistir con otro software independiente, en un entorno común, compartiendo recursos comunes sin detrimento.
- **Interoperabilidad.** Capacidad de dos o más sistemas o componentes para intercambiar información y utilizar la información intercambiada.

5.2.4 USABILIDAD

Capacidad del producto software para ser entendido, aprendido, usado y resultar atractivo para el usuario, cuando se usa bajo determinadas condiciones. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Capacidad para reconocer su adecuación.** Capacidad del producto que permite al usuario entender si el software es adecuado para sus necesidades.

- **Capacidad de aprendizaje.** Capacidad del producto que permite al usuario aprender su aplicación.
- **Capacidad para ser usado.** Capacidad del producto que permite al usuario operarlo y controlarlo con facilidad.
- **Protección contra errores de usuario.** Capacidad del sistema para proteger a los usuarios de hacer errores.
- **Estética de la interfaz de usuario.** Capacidad de la interfaz de usuario de agradar y satisfacer la interacción con el usuario.
- **Accesibilidad.** Capacidad del producto que permite que sea utilizado por usuarios con determinadas características y discapacidades.

5.2.5 FIABILIDAD

Capacidad de un sistema o componente para desempeñar las funciones especificadas, cuando se usa bajo unas condiciones y periodo de tiempo determinados. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Madurez.** Capacidad del sistema para satisfacer las necesidades de fiabilidad en condiciones normales.
- **Disponibilidad.** Capacidad del sistema o componente de estar operativo y accesible para su uso cuando se requiere.
- **Tolerancia a fallos.** Capacidad del sistema o componente para operar según lo previsto en presencia de fallos hardware o software.
- **Capacidad de recuperación.** Capacidad del producto software para recuperar los datos directamente afectados y restablecer el estado deseado del sistema en caso de interrupción o fallo.

5.2.6 SEGURIDAD

Capacidad de protección de la información y los datos de manera que personas o sistemas no autorizados no puedan leerlos o modificarlos. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Confidencialidad.** Capacidad de protección contra el acceso de datos e información no autorizados, ya sea accidental o deliberadamente.
- **Integridad.** Capacidad del sistema o componente para prevenir accesos o modificaciones no autorizados a datos o programas de ordenador.
- **No repudio.** Capacidad de demostrar las acciones o eventos que han tenido lugar, de manera que dichas acciones o eventos no puedan ser repudiados posteriormente.
- **Responsabilidad.** Capacidad de rastrear de forma inequívoca las acciones de una entidad.
- **Autenticidad.** Capacidad de demostrar la identidad de un sujeto o un recurso.

5.2.7 MANTENIBILIDAD

Esta característica representa la capacidad del producto software para ser modificado efectiva y eficientemente, debido a necesidades evolutivas, correctivas o perfectivas. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Modularidad.** Capacidad de un sistema o programa de ordenador (compuesto de componentes discretos) que permite que un cambio en un componente tenga un impacto mínimo en los demás.
- **Reusabilidad.** Capacidad de un activo que permite que sea utilizado en más de un sistema software o en la construcción de otros activos.

- **Analizabilidad.** Facilidad con la que se puede evaluar el impacto de un determinado cambio sobre el resto del software, diagnosticar las deficiencias o causas de fallos en el software, o identificar las partes a modificar.
- **Capacidad para ser modificado.** Capacidad del producto que permite que sea modificado de forma efectiva y eficiente sin introducir defectos o degradar el desempeño.
- **Capacidad para ser probado.** Facilidad con la que se pueden establecer criterios de prueba para un sistema o componente y con la que se pueden llevar a cabo las pruebas para determinar si se cumplen dichos criterios.

5.2.8 PORTABILIDAD

Capacidad del producto o componente de ser transferido de forma efectiva y eficiente de un entorno hardware, software, operacional o de utilización a otro. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Adaptabilidad.** Capacidad del producto que le permite ser adaptado de forma efectiva y eficiente a diferentes entornos determinados de hardware, software, operacionales o de uso.
- **Capacidad para ser instalado.** Facilidad con la que el producto se puede instalar y/o desinstalar de forma exitosa en un determinado entorno.
- **Capacidad para ser reemplazado.** Capacidad del producto para ser utilizado en lugar de otro producto software determinado con el mismo propósito y en el mismo entorno.

5.3 MÉTRICA 3

5.3.1 PRÁCTICAS

Métrica 3 reconoce los siguientes tipos de pruebas, detalladas en su libro de Técnicas y Prácticas, como prácticas:

- Unitarias
- Integración
- Sistema
- Implantación
- Aceptación
- Regresión

Además, en este mismo libro, reconoce las revisiones formales y revisiones técnicas dentro de prácticas.

En Métrica 3, el Plan de pruebas se comienza a definir en el proceso ASI, Análisis de Sistemas de Información, se especifica detalladamente en el proceso DSI, Diseño de Sistemas de Información y se ejecutarán en el proceso CSI, Construcción de Sistemas de Información.

5.3.2 ACTIVIDADES DE PRUEBAS EN PROCESOS MÉTRICA

PROCESO ASI

· Se especifican y justifican los niveles de pruebas, así como el marco general de planificación de cada nivel de prueba.

- Definición de los perfiles implicados.
- Planificación temporal.
- Criterios de verificación y aceptación de cada nivel de pruebas.

- Definición, generación y mantenimiento de casos de prueba.
- Definición de los requisitos relativos al entorno de pruebas.
 - Hardware y software base: SO, SGBD, ...
 - Configuración del entorno: librerías, ficheros, BD, procesos, comunicaciones, necesidades de almacenamiento, configuración de accesos.
 - Procedimientos para la realización de las pruebas y migración de elementos entre entornos.
- Especificación de las pruebas de aceptación del sistema.

PROCESO DSI

- Especificación del entorno de pruebas.
 - Hw, sw y comunicaciones.
 - Restricciones técnicas del entorno.
 - Requisitos de operación y seguridad.
 - Herramientas de prueba relacionadas con la extracción de juegos de ensayo, análisis de resultados, ...
 - Procedimientos de prueba y recuperación, vuelta atrás, ...
- Diseño detallado de los distintos niveles de prueba, especialmente de las pruebas de integración y del sistema.
 - Tipo de prueba y objetivo.
 - Se definen en detalle los casos de prueba, describiendo todas las entradas necesarias para ejecutar la prueba, las relaciones de secuencialidad existentes entre ellas y la salida esperada.
 - Entorno de prueba: herramientas adicionales, condicionantes especiales de ejecución, etc.
 - Criterios de aceptación de la prueba.
 - Análisis y evaluación de resultados.
 - El ajuste del sistema a las necesidades para las que fue creado, de acuerdo a las características del entorno en el que se va a implantar (pruebas de implantación).
 - La respuesta satisfactoria del sistema a los requisitos especificados por el usuario (pruebas de aceptación).

- Planificación de las pruebas.

- Determinando los distintos perfiles implicados en la preparación y ejecución de las pruebas y en la evaluación de los resultados, así como el tiempo estimado para la realización de cada uno de los niveles de prueba, de acuerdo a la estrategia de integración establecida

PROCESO CSI

- Ejecución de las pruebas unitarias.
- Ejecución de las pruebas de integración.
- Ejecución de las pruebas del sistema.

PROCESO IAS

- Ejecución de las pruebas de implantación.
- Ejecución de las pruebas de aceptación.

PROCESO MSI (si se produce una petición de cambio y se aprueba, una vez realizado)

- Ejecución de las pruebas de regresión.

Además, el Aseguramiento de la Calidad es una de las 4 interfaces de métrica cuyo objetivo es proporcionar un marco común de referencia para la definición y puesta marcha de planes específicos de aseguramiento de calidad aplicables a proyectos concretos.

5.4 ISTQB

International Software Testing Qualifications Board, ISTQB. Es un Comité internacional que suministra planes de estudios y glosarios para distintos niveles sobre los que se definen las guías para la acreditación y evaluación de los profesionales de pruebas.

6 TIPOS DE PRUEBA

Un grupo de actividades de pruebas pueden tener por objetivo verificar el sistema de software (o parte del sistema) en base a un motivo u objetivo específico para probar.

Un tipo de prueba se centra en un objetivo de prueba en particular, en función del objeto de prueba se encuentra la siguiente clasificación:

- **Pruebas funcionales:** Una función a realizar por el software.
- **Pruebas no Funcionales:** Una característica de calidad no funcional, tales como la fiabilidad o la usabilidad.
- **Pruebas estructurales:** La estructura o arquitectura del software o sistema.
- **Pruebas de Confirmación o re-testing** Confirmar que se han solucionado los defectos.
- **Pruebas de Regresión:** Están asociadas a la fase de mantenimiento del sistema y su objetivo es comprobar que los cambios efectuados sobre un componente del sistema de información no introducen un comportamiento no deseado o errores adicionales en los componentes no modificados.

7 NIVELES DE PRUEBA

7.1 PRUEBAS DE COMPONENTES O UNITARIAS

Las pruebas de componente, pruebas unitarias, pruebas de módulo o pruebas de programa, tienen por objeto localizar defectos y comprobar el funcionamiento de módulos de software, programas, objetos, clases, etc., que pueden probarse por separado.

Pueden realizarse de manera aislada del resto del sistema, en función del contexto del ciclo de vida de desarrollo y del sistema. Para ello pueden utilizarse stubs, controladores y simuladores.

En general, las pruebas de componente se llevan a cabo mediante el acceso al código objeto de las pruebas y con el soporte de un entorno de desarrollo, como por ejemplo un marco de trabajo para

pruebas unitarias o una herramienta de depuración. En la práctica, las pruebas de componente generalmente cuentan con la participación del programador que escribió el código. En general, los defectos se corrigen en el momento en que se detectan, sin gestionarlos formalmente.

Son objetos de prueba típicos: componentes, programas, conversión de datos / programas de migración y módulos de bases de datos.

7.2 PRUEBAS DE INTEGRACIÓN

Las pruebas de integración se ocupan de probar las interfaces entre los componentes, las interacciones con distintas partes de un mismo sistema, como el sistema operativo, el sistema de archivos y el hardware, y las interfaces entre varios sistemas.

Para las pruebas de integración se establecen dos estrategias:

- Integración incremental.
- Integración no incremental o Big-Bang.

Pruebas de integración Incremental. En las pruebas unitarias, cuando se prueba cada módulo individualmente es necesario crear módulos auxiliares (resguardos según Pressman) que simulen las acciones de los módulos invocados por el que se está probando, y módulos «conductores» (controladores según Pressman) para establecer las precondiciones necesarias, llamar al módulo objeto de la prueba y examinar los resultados de la prueba.

Las posibles estrategias de integración incremental son:

- Estrategia top-down**, descendente, o de arriba a abajo. Consiste en empezar la integración y la prueba por los módulos que están en los niveles superiores. Se necesitan módulos auxiliares que sustituyan a los módulos que se llaman desde los de alto nivel.
- Estrategia bottom-up**, ascendente, o de abajo a arriba. Consiste en empezar la integración y prueba por los módulos que están en los niveles inferiores. En este caso se necesitan módulos conductores que simulen las llamadas a los que se están probando.

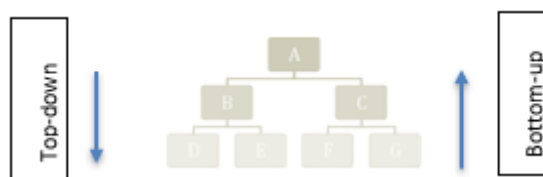


Imagen 8 Top-down y bottom-up

Pruebas de integración no incremental o Big-Bang. Consiste en integrar y probar todo al mismo tiempo. Con esta prueba se detectan muchos errores, pero es complicado identificar las causas que los producen.

Son objetos de prueba típicos: subsistemas, implementación de base de datos, infraestructura, interfaces, configuración del sistema y datos de configuración.

7.3 PRUEBAS DE SISTEMA

Las pruebas de sistema se refieren al comportamiento de todo un sistema/producto. En las pruebas de sistema, el entorno de pruebas debe coincidir en la máxima medida posible con el objetivo final o con el

entorno de producción a fin de minimizar el riesgo de no identificar fallos específicos del entorno durante las pruebas.

Las pruebas de sistema pueden incluir pruebas basadas en el riesgo y/o en especificaciones de requisitos, procesos de negocio, casos de uso u otras descripciones de texto de alto nivel o modelos de comportamiento del sistema, interacciones con el sistema operativo y recursos del sistema.

Las pruebas de sistema deben estudiar los requisitos funcionales y no funcionales del sistema y las características de calidad de los datos. Las pruebas de sistema de los requisitos funcionales utilizan técnicas basadas en la especificación o técnicas de caja negra, más apropiadas para el aspecto del sistema a probar.

Dentro de este nivel de prueba se encuentran:

- **Pruebas de seguridad.** Intenta verificar que los mecanismos de protección incorporados en el sistema lo protegerán, de hecho, de accesos impropios.
- **Pruebas de rendimiento.** Para sistemas de tiempo real es inaceptable el software que proporciona las funciones requeridas, pero no se ajusta a los requisitos de rendimiento.
- **Pruebas de recuperación.** Muchos sistemas deben recuperarse de los fallos y continuar el proceso en un tiempo previamente especificado.
- **Pruebas de resistencia o stress.** Ejecuta un sistema de forma que demande recursos en cantidad, frecuencia o volúmenes anormales. Esencialmente, el responsable de la prueba intenta sobrecargar el sistema.

Son objetos de prueba típicos: manuales de sistema, usuario y funcionamiento, configuración del sistema y datos de configuración.

7.4 PRUEBAS DE ACEPTACIÓN

Las pruebas de aceptación son a menudo responsabilidad de los clientes o usuarios de un sistema, a pesar de que también pueden participar otros implicados.

El objetivo de las pruebas de aceptación es crear confianza en el sistema, partes del sistema o ciertas características no funcionales del sistema. El objetivo principal de las pruebas de aceptación no es localizar defectos, sino que es evaluar la buena disposición de un sistema para su despliegue.

En general, las pruebas de aceptación pueden adoptar, entre otras, las siguientes formas:

Pruebas de aceptación de usuario

En general, verifican la idoneidad de uso del sistema por parte de los usuarios de negocio.

Pruebas de aceptación operativas

La aceptación del sistema por parte de los administradores del sistema.

Pruebas de aceptación contractual y normativa

Las pruebas de aceptación contractual toman como base los criterios de aceptación previstos en un contrato para producir un software desarrollado a medida. Los criterios de aceptación deberán establecerse en el momento en que las partes aceptan contraer dicho contrato. Las pruebas de aceptación de regulación toman como base la normativa aplicable, tales como normativa, internacional y gubernamental.

Pruebas alfa y beta (o de campo)

Los desarrolladores de software comercial de distribución masiva a menudo quieren obtener información de retorno (feedback) de clientes potenciales o existentes en su mercado antes de poner a la venta un producto de software. Las pruebas alfa se llevan a cabo en el emplazamiento de la organización de

desarrollo, pero no las realiza el equipo de desarrollo. Las pruebas beta, o pruebas de campo, las realizan los clientes o clientes potenciales en sus propias instalaciones.

Son objetos de prueba típicos: procesos de negocio en sistema completamente integrado, procesos operativos y de mantenimiento, procedimientos de usuario, formularios, informes, datos de configuración.

8 TÉCNICAS DE PRUEBAS

Existen distintas técnicas para el diseño de pruebas definidas dentro de las medidas analíticas de aseguramiento de la calidad, según ISTQB. Se clasifican en: técnicas estáticas, aquellas que no implican la ejecución del software y técnicas dinámicas, si su aplicación conlleva que el software sea ejecutado.

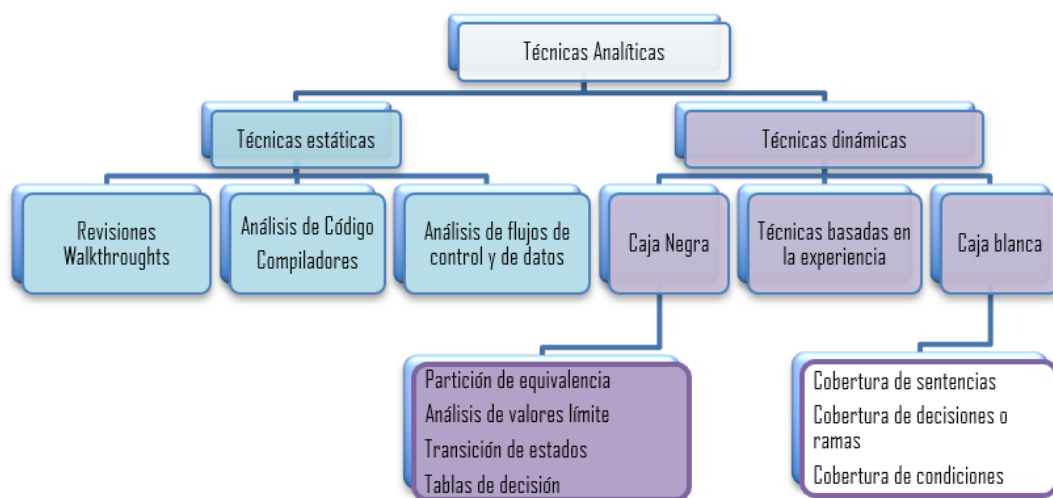


Imagen 9 Técnicas Analíticas o Medidas QA – Clasificación ISTQB

6.1. TÉCNICAS ESTÁTICAS

Las actividades de control de la calidad se pueden clasificar en:

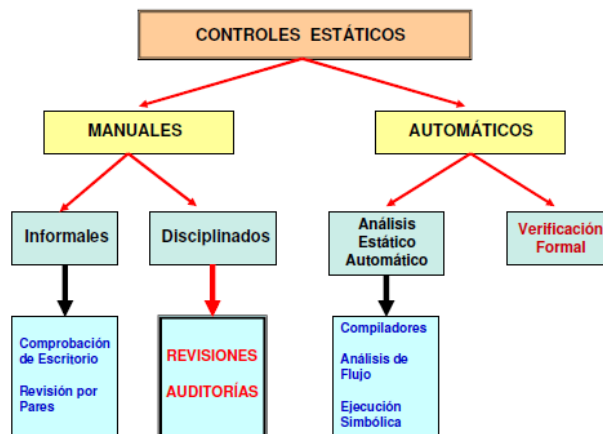


Imagen 10 Detalle Técnicas Estáticas (distintas clasificaciones)

Revisiones formales o disciplinadas:

Auditorías: investigación para determinar el grado de cumplimiento y la adecuación de los procedimientos, estándares u otros requisitos establecidos, y la efectividad de la implementación realizada. Hay 3 tipos: auditoría del producto (según requisito), auditoría del proceso (de desarrollo) del proyecto o su gestión y auditoría del sistema de garantía de calidad.

Revisiones: reunión formal en la que se presenta el estado actual de un proyecto a los interesados. Se realizan al principio del desarrollo (auditoría al final). Pueden ser formales (públicas, se informa de los resultados por escrito) o informales y hay dos tipos: tipo inspecciones (se revisa un checklist de comprobación) y tipo walkthroughs (simulación del funcionamiento con casos de prueba y ejemplos).

Según su alcance, el Plan General de Garantía de Calidad del Consejo Superior de Administración Electrónica, distingue entre:

- Revisiones Mínimas (RM – examen de las características + visibles de los documentos)
- Revisiones Técnicas Formales (RTF – evaluar que los documentos son conformes a las especificaciones)
- Inspecciones Detalladas (ID - detección de cualquier tipo de defectos que puedan afectar a un documento obtenido al final de una fase de desarrollo, la diferencia con RTF es que RTF comprueba adecuación e ID busca defectos).

En Métrica V3, se indica otra clasificación.

- Revisiones Formales (rigurosas: no se aplican siempre porque puede bloquear el proyecto, detectan defectos: que no satisface especificaciones)
- Revisiones Técnicas (comprobar adecuación con especificaciones). Pressman sólo indica
- Revisiones Técnicas Formales (detectar errores, no según especificaciones o estándares, sw uniforme, proyecto manejable).

Verificación formal: demostrar matemáticamente la corrección de un programa según especificaciones. Alto coste sólo para sistemas críticos.

Análisis de código y Compiladores

Con el uso de herramientas para la realización de análisis estático (compiladores, analizadores) el código del programa puede ser objeto de inspección sin ser ejecutado. Con el uso de herramientas se puede realizar el análisis estático de un programa con un esfuerzo menor que el necesario para una inspección. Las métricas aplicadas al análisis estático a través de analizadores de código pueden ser utilizadas para evaluar la complejidad estructural, conduciendo o una estimación del esfuerzo en pruebas esperado. Los compiladores permitirán conocer la corrección de la sintaxis del código sin necesidad de ejecución.

Análisis de control de flujo y de datos

El diagrama de control de flujo presenta el flujo del programa y permite la detección de ramas muertas y código inalcanzable. Las anomalías en los datos, como definiciones y conversiones, se detectan realizando un análisis del flujo de datos.

6.2. TÉCNICAS DINÁMICAS

Son aquellas que requieren la ejecución del objeto que se está probando o de un modelo del mismo. Para cada nivel de pruebas encontramos aplicación de técnicas dinámicas de prueba, recordando los niveles de prueba:

- Prueba modular, unitaria o de componentes: se prueban los módulos aislados usando métodos de caja negra o pruebas funcionales (probar la funcionalidad sin tener en cuenta la estructura) y de caja blanca (prueba la estructura del módulo).
- Prueba de integración: comprobar que las interfaces definidas entre los distintos módulos son correctas, se pueden hacer: top-down (empezar integrando por arriba), bottom-up o big bang (todo a la vez).
- Prueba de sistema: se prueba los RF y los RFN una vez integrados todos los módulos.
- Pruebas de aceptación: demostrar al usuario, en un entorno real, que satisface sus requisitos.
- Pruebas de regresión: probar que la calidad es mínimo como la de la versión anterior.

Las técnicas dinámicas se clasifican en tres tipos:

Métodos basados en la estructura o de Caja Blanca: la estructura Interna del objeto de prueba es utilizada para diseñar los casos de prueba (código/sentencias, menús, llamados, etc.). El porcentaje de cobertura es medido y utilizado como fuente para la creación de casos de prueba adicionales

- Pruebas de sentencias y cobertura
- Pruebas de decisiones y cobertura

Métodos basados en la especificación o de Caja Negra: el objeto de prueba ha sido seleccionado de acuerdo con el modelo funcional de software.

- Partición de equivalencia
- Análisis de valores límite
- Prueba de tabla de decisión
- Pruebas de transición de estado
- Pruebas de caso de uso

Métodos basados en la experiencia: el conocimiento y experiencia respecto de los objetos de prueba y su entorno son las fuentes para el diseño de casos de prueba, constituye otra fuente de información.

- Predicción de error
- Pruebas exploratorias

8.1 PRUEBAS DE CAJA BLANCA

Cobertura de sentencias

Diseño de casos de prueba suficientes para que cada sentencia en el programa se ejecute, al menos, una vez.

$$COBERTURA SENTENCIA = \frac{N.º SENTENCIAS EJECUTADAS EN LA PRUEBA}{N.º TOTAL SENTENCIAS}$$

Cobertura de condición

Diseño de casos de prueba suficientes para que cada condición en una decisión tenga una vez resultado verdadero y otro falso.

$$COBERTURA CONDICIÓN = \frac{N.º CONDICIONES EJECUTADAS EN LA PRUEBA}{N.º TOTAL CONDICIONES}$$

Cobertura Decisión/Condición:

Se escriben casos de prueba suficientes para que cada condición en una decisión tome todas las posibles salidas, al menos una vez, y cada decisión tome todas las posibles salidas, al menos una vez.

Cobertura de Condición Múltiple:

Se escriben casos de prueba suficientes para que todas las combinaciones posibles de resultados de cada condición se invoquen al menos una vez.

El 100% de cobertura de sentencia no implica el 100% de cobertura de condición.

Cobertura de Caminos:

Se escriben casos de prueba suficientes para que se ejecuten todos los caminos de un programa. Entendiendo camino como una secuencia de sentencias encadenadas desde la entrada del programa hasta su salida.

$$COBERTURA\ CAMINOS = N.^{\circ} CAMINOS\ EJECUTADOS\ EN\ LA\ PRUEBA / N.^{\circ} TOTAL\ CAMINOS$$

8.2 PRUEBAS DE CAJA NEGRA

Partición de Equivalencia (EP) (Clases de equivalencia)

Permite seleccionar datos de prueba y se pueden aplicar a todos los niveles de pruebas. Los valores de entrada del software o del sistema se dividen en los grupos que se espera demuestren un comportamiento similar. Las particiones de equivalencia (Clases) son aplicables a: Datos válidos: valores que deben aceptarse Datos inválidos: valores que deben rechazarse Valores de salida, internos, relativos al tiempo, parámetros de interfaz. Está basado en el concepto de “Aceptación” (Assumption), esto es, si un valor funciona, el resto también lo hará.

Pruebas de Análisis de valores límite (BVA)

Permite seleccionar datos de prueba y se pueden aplicar a todos los niveles de pruebas. Los límites constituyen un área en el que las pruebas tenderán a incluir defectos. Presenta una mayor detección de defecto que el método de Clases de Equivalencia. Los valores máximos y mínimos de una partición son sus valores límites. Las pruebas pueden diseñarse para cubrir valores límite tanto válidos como inválidos., de modo que, al diseñar casos de pruebas, se selecciona una prueba por valor límite.

Tablas de decisión

A través de tablas, se representan los requisitos del sistema que contienen condiciones lógicas y de documentar el diseño del sistema interno. Se deben incluir combinación de salidas, situaciones o eventos. Ayuda a asegurar que no se pasa por alto ninguna combinación importante. Como es imposible realizar todas las combinaciones, estas tablas de decisión ayudan a identificar un buen subconjunto de datos de prueba.

Pruebas de transición de estados

Estas técnicas ayudan al diseño de la prueba ya que los casos de pruebas podrán cubrir: la secuencia estados, cualquier estado, cualquier transición y la secuencia de transiciones.

Pruebas de caso de uso

El caso de uso contiene la descripción de cómo debe usarse el sistema, especificado desde la perspectiva de usuarios de negocio. Esta técnica contempla su uso en el diseño de la prueba de modo que, se contemplen los procesos generales, alternativas, pre-condiciones, post-condiciones y roles aplicables.

9 HERRAMIENTAS DE PRUEBAS DE SOFTWARE

	OPEN SOURCE	COMERCIALES
GESTIÓN DE PRUEBAS	○ Bugzilla Testopia ○ FitNesse ○ qaManager ○ qaBook ○ RTH (open source) ○ Salome-tmf ○ Squash TM ○ Test Environment Toolkit ○ TestLink ○ Testitool ○ XQual Studio ○ Radi-testdir ○ Data Generator	○ HP Quality Center/ALM ○ QA Complete ○ qaBook ○ T-Plan Professional ○ SMARTS ○ QAS.Test Case Studio ○ PractiTest ○ SpiraTest ○ TestLog ○ ApTest Manager ○ Zephyr
PRUEBAS FUNCIONALES	○ Selenium ○ Cypress ○ Appium (Mobile Web Apps, native or hybrid) ○ Postman ○ Soapui ○ NUnit (○ Watir (Pruebas de aplicaciones web en Ruby) ○ WatiN (Pruebas de aplicaciones web en .Net) ○ Capedit ○ Canoo WebTest ○ Solex ○ Imprimatur ○ SAMIE ○ ITP ○ WET ○ WebInject ○	○ QuickTest Pro ○ Rational Robot ○ Sahi ○ SoapTest ○ Test Complete ○ QA Wizard ○ Squish ○ vTest ○ Internet Macros
PRUEBAS DE CARGA Y RENDIMIENTO	○ FunkLoad ○ FWPTT load testing ○ loadUI ○ JMeter	○ HP LoadRunner ○ LoadStorm ○ NeoLoad ○ WebLOAD Professional ○ Forecast ○ ANTS – Advanced .NET Testing System ○ Webserver Stress Tool ○ Load Impact
PRUEBAS UNITARIAS	○ XUnit: JUnit, NUnit, RUnit, PHPUnit... ○ TestNG: Creado para suplir algunas deficiencias en JUnit. ○ NUnit (.Net) ○ JTiger: Basado en anotaciones, como TestNG. ○ CPPUnit ○ Visual Studio UnitTesting	

Tabla 1 Herramientas